

FLIGHT BOOKING SYSTEM – PROJECT DOCUMENTATION

(Spring Boot + MySQL)

1. Introduction

This document describes the design, architecture, features, validation rules, database structure, and API details for the Flight Booking System backend application. The system provides APIs for searching flights, booking tickets, retrieving PNR details, viewing booking history, and canceling bookings. It is designed using a layered Spring Boot architecture and uses MySQL as the database.

2. Features

2.1 User Features

- Search flights by source, destination, and date
- Book tickets with passenger details and seat selection
- Retrieve ticket details using PNR
- View booking history using email
- Cancel booking (permitted only if the flight departure is more than 24 hours away)

2.2 Admin Features

- Add flight inventory and schedules
- Automatic maintenance of seat availability during bookings and cancellations

3. Architecture

The project follows a layered architecture:

- Controller Layer
- Service Layer
- Repository Layer (JPA)
- Entity/Model Layer
- Global Exception Handler
- Validation Layer (using Jakarta Bean Validation)

Flow:

Client → Controller → Service → Repository → Database

4. Technology Stack

- Java 17
- Spring Boot
- Spring Web
- Spring Validation
- Spring Data JPA
- MySQL
- Lombok
- JUnit 5 + Mockito
- Maven

5. Database Design

5.1 Flight Table

Fields:

- id (Primary Key)
- flightNumber (Unique)
- airline
- fromCity
- toCity
- departureTime
- arrivalTime
- totalSeats
- availableSeats
- price

Constraints:

- fromCity cannot be the same as toCity
- arrivalTime must be after departureTime
- flightNumber must be unique

5.2 Booking Table

Fields:

- id
- pnr (Unique)
- name
- email
- seatsBooked
- mealType
- flight_id (Foreign Key)

5.3 Passenger Table

Fields:

- id
- name
- gender (M, F, O)
- age (1-120)
- seatNumber (Format: A1, B2, C10, etc.)
- booking_id (Foreign Key)

Constraints:

- Unique(seatNumber, booking_id)
- No duplicate passengers with same name + age + gender
- Seat number must match the pattern `^[A-Z][0-9]+$`

6. Validation Rules

6.1 Flight Validations

- airline: required
- flightNumber: required, unique
- fromCity and toCity: must be different
- departureTime and arrivalTime: arrival must be later
- totalSeats: minimum 1
- price: cannot be negative

6.2 Passenger Validations

- name: required
- gender: must be M, F, or O
- age: must be between 1 and 120
- seatNumber: must match format such as A1, B5
- seat must not be duplicated within the same booking
- no duplicate passengers allowed (same name + age + gender)

6.3 Booking Validations

- name: required
- email: must be valid
- seats: must be positive
- passengers: count must match number of seats
- mealType: VEG or NON_VEG
- cancellation allowed only if departure is more than 24 hours away

7. API Documentation

Base URL: localhost:8080/api/v1.0/flight

7.1 Add Flight (Admin)

POST /airline/inventory/add

Sample Request and outputs:

The screenshot displays a REST client interface with a POST request to `localhost:8080/api/v1.0/flight/airline/inventory/add`. The request body is a JSON object with the following fields: `airline` (AirIndia), `flightNumber` (AI250), `fromCity` (DEL), `toCity` (BOM), `departureTime` (2025-12-10T09:00:00), `arrivalTime` (2025-12-10T11:30:00), `totalSeats` (150), `availableSeats` (0), and `price` (4500). The response status is 200 OK, and the response body is a JSON object with the following fields: `id` (1), `airline` (AirIndia), `flightNumber` (AI250), `fromCity` (DEL), `toCity` (BOM), `departureTime` (2025-12-10T09:00:00), `arrivalTime` (2025-12-10T11:30:00), `totalSeats` (150), `availableSeats` (150), `price` (4500.0), `differentCities` (true), and `validTimes` (true).

```
POST localhost:8080/api/v1.0/flight/airline/inventory/add

{
  "airline": "AirIndia",
  "flightNumber": "AI250",
  "fromCity": "DEL",
  "toCity": "BOM",
  "departureTime": "2025-12-10T09:00:00",
  "arrivalTime": "2025-12-10T11:30:00",
  "totalSeats": 150,
  "availableSeats": 0,
  "price": 4500
}

200 OK

{
  "id": 1,
  "airline": "AirIndia",
  "flightNumber": "AI250",
  "fromCity": "DEL",
  "toCity": "BOM",
  "departureTime": "2025-12-10T09:00:00",
  "arrivalTime": "2025-12-10T11:30:00",
  "totalSeats": 150,
  "availableSeats": 150,
  "price": 4500.0,
  "differentCities": true,
  "validTimes": true
}
```

Invalid scenarios:

POST localhost:8080/api/v1.0/flight/airline/inventory/add

Docs Params Auth Headers (9) Body Scripts Settings

raw JSON

```
1 {
2   "id": 1,
3   "airline": "AirIndia",
4   "flightNumber": "AI250",
5   "fromCity": "DEL",
6   "toCity": "BOM",
7   "availableSeats": 150
8 }
```

Body 400 Bad Request

JSON Preview Debug with AI

```
1 {
2   "departureTime": "must not be null",
3   "arrivalTime": "must not be null",
4   "totalSeats": "must be greater than or equal to 1"
5 }
```

POST localhost:8080/api/v1.0/flight/airline/inventory/add

Docs Params Auth Headers (9) Body Scripts Settings

raw JSON

```
1 {"flightNumber": "AI250"}
2
```

Body 400 Bad Request

JSON Preview Debug with AI

```
1 {
2   "departureTime": "must not be null",
3   "arrivalTime": "must not be null",
4   "toCity": "must not be null",
5   "airline": "must not be blank",
6   "fromCity": "must not be null",
7   "differentCities": "fromCity and toCity cannot be same",
8   "totalSeats": "must be greater than or equal to 1"
9 }
```

POST localhost:8080/api/v1.0/flight/airline/inventory/add

Docs Params Auth Headers (9) Body Scripts Settings

raw JSON

```
1 {
2   "airline": "AI",
3   "flightNumber": "AI250",
4   "fromCity": "DEL",
5   "toCity": "BOM",
6   "departureTime": "2025-12-10T10:00:00",
7   "arrivalTime": "2025-12-10T09:00:00",
8   "totalSeats": 100,
9   "price": 5000
10 }
```

Body 400 Bad Request

JSON Preview Debug with AI

```
1 {
2   "validTimes": "Arrival time must be after departure time"
3 }
```

POST localhost:8080/api/v1.0/flight/airline/inventory/add

Docs Params Auth Headers (9) Body Scripts Settings

raw JSON

```
1 {
2   "airline": "AI",
3   "flightNumber": "AI250",
4   "fromCity": "DEL",
5   "toCity": "DEL",
6   "departureTime": "2025-12-10T10:00:00",
7   "arrivalTime": "2025-12-10T11:00:00",
8   "totalSeats": 100,
9   "price": 5000
10 }
```

Body 400 Bad Request

JSON Preview Debug with AI

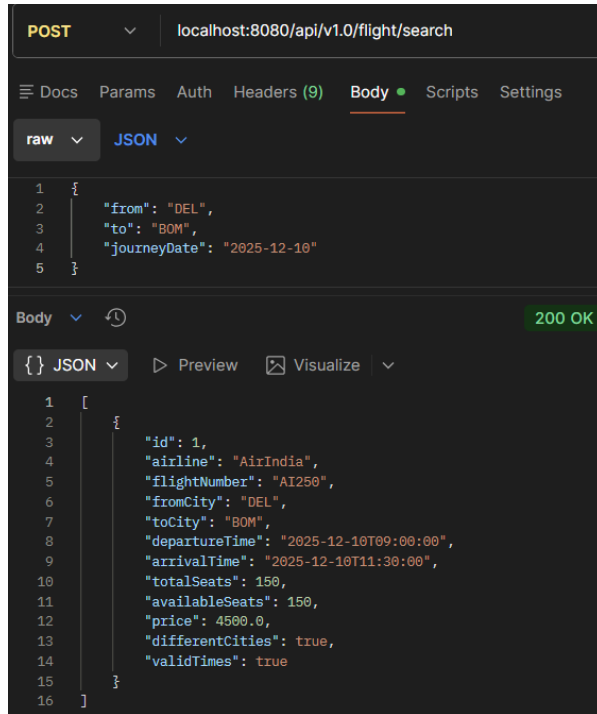
```
1 {
2   "differentCities": "fromCity and toCity cannot be same"
3 }
```

7.2 Search Flights

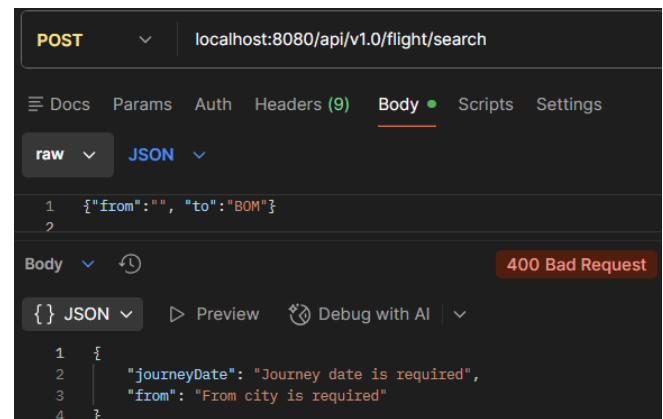
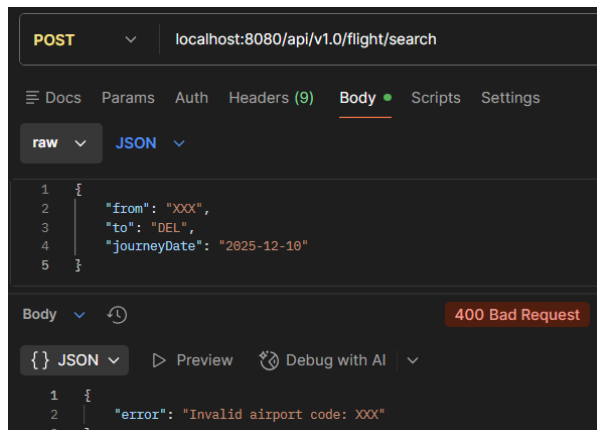
POST /search

Sample Request and output:

Valid scenario:



Invalid scenarios:



7.3 Book Ticket

POST /booking/{flightId}

Sample Request and output:

POST localhost:8080/api/v1.0/flight/booking/1

Docs Params Auth Headers (9) Body Scripts Settings

raw JSON

```
1 {
2   "name": "Test User",
3   "email": "test@user.com",
4   "seats": 2,
5   "mealType": "VEG",
6   "passengers": [
7     {
8       "name": "A",
9       "gender": "M",
10      "age": 30,
11      "seatNumber": "A1"
12    },
13    {
14      "name": "B",
15      "gender": "F",
16      "age": 28,
17      "seatNumber": "A2"
18    }
19  ]
20 }
```

Body 200 OK

Raw Preview Visualize

1 PNR66240

Invalid scenarios:

POST localhost:8080/api/v1.0/flight/booking/1

Docs Params Auth Headers (9) Body Scripts Settings

raw JSON

```
1 {
2   "name": "",
3   "email": "abc",
4   "seats": 1,
5   "mealType": "VEG",
6   "passengers": [
7     {
8       "name": "P",
9       "gender": "M",
10      "age": 20,
11      "seatNumber": "A1"
12    }
13  ]
14 }
```

Body 400 Bad Request

{ JSON Preview Debug with AI

```
1 {
2   "name": "Name is required",
3   "email": "Invalid email format"
4 }
```

POST localhost:8080/api/v1.0/flight/booking/1

Docs Params Auth Headers (9) Body Scripts Settings

raw JSON

```
1 {
2   "name": "X",
3   "email": "x@x.com",
4   "seats": 2,
5   "mealType": "VEG",
6   "passengers": [
7     {
8       "name": "A",
9       "gender": "M",
10      "age": 20,
11      "seatNumber": "A1"
12    }
13  ]
14 }
```

Body 400 Bad Request

{ JSON Preview Debug with AI

```
1 {
2   "error": "Number of passengers must match seats booked"
3 }
```

```
POST localhost:8080/api/v1.0/flight/booking/1

{
  "name": "X",
  "email": "x@x.com",
  "seats": 2,
  "mealType": "VEG",
  "passengers": [
    {
      "name": "A",
      "gender": "M",
      "age": 20,
      "seatNumber": "A1"
    },
    {
      "name": "A",
      "gender": "M",
      "age": 20,
      "seatNumber": "A2"
    }
  ]
}
```

Body 400

```
{
  "error": "Duplicate passenger detected: A"
}
```

```
POST localhost:8080/api/v1.0/flight/booking/1

{
  "seats": 2,
  "name": "X",
  "email": "x@x.com",
  "mealType": "VEG",
  "passengers": [
    {
      "name": "A",
      "gender": "M",
      "age": 20,
      "seatNumber": "A1"
    },
    {
      "name": "B",
      "gender": "F",
      "age": 22,
      "seatNumber": "A1"
    }
  ]
}
```

Body 400 B

```
{
  "error": "Seat already booked: A1"
}
```

7.4 Get Ticket Details

GET /ticket/{pnr}

```
GET localhost:8080/api/v1.0/flight/ticket/PNR66240

{
  "pnr": "PNR66240",
  "name": "Test User",
  "seatsBooked": 2
}
```

Body 200 OK • 69 ms • 662 B

```
{
  "id": 1, "pnr": "PNR66240", "name": "Test User", "email": "test@user.com", "seatsBooked": 2,
  "mealType": "VEG", "flight": {
    "id": 1, "airline": "AirIndia", "flightNumber": "AI250",
    "fromCity": "DEL", "toCity": "BOM", "departureTime": "2025-12-10T09:00:00",
    "arrivalTime": "2025-12-10T11:30:00", "totalSeats": 150, "availableSeats": 148, "price": 4500.0,
    "differentCities": true, "validTimes": true
  }, "passengers": [
    { "id": 1, "name": "A", "gender": "M", "age": 30, "seatNumber": "A1" },
    { "id": 2, "name": "B", "gender": "F", "age": 28, "seatNumber": "A2" }
  ]
}
```

7.5 Booking History

GET /booking/history/{email}

GET localhost:8080/api/v1.0/flight/booking/history/test@user.com

Body

200 OK • 30 ms • 708 B

```
1 [{"id":1,"pnr":"PNR66240","name":"Test User","email":"test@user.com","seatsBooked":2,
2   "mealType":"VEG","flight":{"id":1,"airline":"AirIndia","flightNumber":"AI250",
3   "fromCity":"DEL","toCity":"BOM","departureTime":"2025-12-10T09:00:00",
4   "arrivalTime":"2025-12-10T11:30:00","totalSeats":150,"availableSeats":148,"price":4500.0,
   "differentCities":true,"validTimes":true},"passengers":[{"id":1,"name":"A","gender":"M",
   "age":30,"seatNumber":"A1"},{"id":2,"name":"B","gender":"F","age":28,"seatNumber":"A2"}]}
```

7.6 Cancel Booking

DELETE

DELETE localhost:8080/api/v1.0/flight/booking/cancel/PNR66240

Params

Query Params

Key	Value	Description
Key	Value	Description

Body

200 OK • 74 ms

```
1 Booking cancelled successfully: PNR66240
```

DELETE localhost:8080/api/v1.0/flight/booking/cancel/PNR6624

Params

Query Params

Key	Value	Description
Key	Value	Description

Body

404 Not Found

```
1 {
2   "error": "Invalid PNR"
3 }
```


8. Exception Handling

All exceptions are handled using the GlobalErrorHandler class.

Error format:

```
{"error": "<message>"}
```

Common errors:

- Invalid airport code
- Invalid PNR
- Seat unavailable
- Duplicate passenger
- Validation failures (field-level errors)
- Cannot cancel within 24 hours
- Business rule violations

9. Testing and Coverage

Tools used:

- JUnit 5
- Mockito
- MockMvc
- JaCoCo

Coverage Achieved:

- 90 percent instruction coverage
- 75 percent branch coverage

FlightBooking												
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.chubb.service		90%		75%	13	35	9	113	2	13	0	1
com.chubb.controller		81%		n/a	1	9	2	10	1	9	0	1
com.chubb.entity		94%		100%	0	13	2	23	0	8	0	5
com.chubb		93%		75%	2	11	2	24	1	9	0	2
com.chubb.exception		100%		n/a	0	3	0	6	0	3	0	3
com.chubb.request		100%		n/a	0	3	0	3	0	3	0	3
Total	70 of 818	91%	12 of 58	79%	16	74	15	179	4	45	0	15

10. Running the Project

1. Create MySQL database: CREATE DATABASE flight_db;
2. Update application.properties with MySQL credentials.
3. Run the application: mvn spring-boot:run
4. Test using Postman or any REST client.

11. Conclusion

The Flight Booking System backend implements a complete and stable flight management workflow with strong validation, proper modular architecture, and robust exception handling.

It is suitable for academic submissions, interviews, training evaluations, and foundational backend development.